

01. Introduzione a Python e Intelligenza Artificiale

Benvenuto

Questo manuale è stato progettato per guidarvi attraverso i concetti fondamentali di Python e mostrarvi come utilizzarlo nel contesto dell'intelligenza artificiale (AI).

Obiettivi della Lezione

- Comprendere le basi del linguaggio Python.
- Familiarizzare con l'ambiente di sviluppo.
- Esplorare l'applicazione di Python nell'intelligenza artificiale.

1. Perché Python?

Python è uno dei linguaggi di programmazione più utilizzati grazie a:

- **Semplicità**: Una sintassi chiara e leggibile.
- **Versatilità**: Utilizzabile in ambiti diversi come web development, data science, automazione e intelligenza artificiale.
- **Comunità**: Una vasta comunità di supporto e risorse.

Python è il linguaggio scelto per questa specializzazione in AI grazie alla disponibilità di librerie come TensorFlow, PyTorch, scikit-learn e soprattutto LangChain.

2. Concetti Fondamentali

Variabili e Tipi di Dati

In Python, una variabile è uno spazio di memoria utilizzato per memorizzare dati. Esempio:

```
x = 5          # Intero
pi = 3.14      # Float
nome = "Alice" # Stringa
```

Operatori

Python supporta operatori aritmetici, di confronto e logici. Ad esempio:

```
somma = 2 + 3
```

Strutture di Controllo

Le strutture di controllo come i cicli `for`, `while` e le condizioni `if` permettono di controllare il flusso del programma.

```
if x > 0:
    print("Positivo")
else:
    print("Non positivo")
```

nota l'indentazione significativa!

Funzioni

Le funzioni sono blocchi di codice che eseguono compiti specifici.

```
def saluta(nome):
    print(f"Ciao, {nome}!")

saluta("Mario")
```

Cicli

I cicli `for` e `while` sono fondamentali nella programmazione per eseguire blocchi di codice in modo ripetitivo. Ecco una guida breve ma esaustiva.

Il ciclo for

Il ciclo `for` viene utilizzato quando si conosce il numero di iterazioni da eseguire. Si itera attraverso sequenze, come liste, stringhe o intervalli.

Sintassi base

```
for variabile in sequenza:
    # codice da eseguire
```

Esempio 1: Iterare su una lista

```
frutti = ["mela", "banana", "ciliegia"]
for frutto in frutti:
    print(frutto)
```

Output:

```
mela
banana
```

Esempio 2: Usare range()

Il metodo `range()` genera una sequenza di numeri.

```
for i in range(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

Esempio 3: Iterare con range() personalizzato

```
for i in range(2, 10, 2): # Inizia da 2, arriva fino a 10 (escluso), con  
    passo di 2  
    print(i)
```

Output:

```
2  
4  
6  
8
```

Il ciclo while

Il ciclo `while` viene utilizzato quando il numero di iterazioni non è noto in anticipo. Continua finché una condizione rimane vera.

Sintassi base

```
while condizione:  
    # codice da eseguire
```

Esempio 1: Contatore semplice

```
contatore = 0  
while contatore < 5:
```

```
print(contatore)
contatore += 1 # Incremento del contatore
```

Output:

```
0
1
2
3
4
```

Esempio 2: Condizione basata sull'input

```
risposta = ""
while risposta != "esci":
    risposta = input("Scrivi qualcosa (digita 'esci' per terminare): ")
    print(f"Hai scritto: {risposta}")
```

Confronto tra for e while

Caratteristica	for	while
Quando usarlo	Quando il numero di iterazioni è noto in anticipo	Quando il numero di iterazioni è sconosciuto
Iterazioni	Attraversa una sequenza o un intervallo	Basato su una condizione
Esempio comune	Iterare su liste o range	Loop fino a una condizione basata su input

Comandi utili nei cicli

break

Interrompe il ciclo immediatamente.

```
for i in range(10):
    if i == 5:
        break
    print(i)
```

Output:

```
0
1
2
```

continue

Salta l'iterazione corrente e passa alla successiva.

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Output:

```
0  
1  
3  
4
```

else nei cicli

Viene eseguito se il ciclo termina senza che sia stato eseguito un **break**.

```
for i in range(5):  
    print(i)  
else:  
    print("Ciclo completato!")
```

Output:

```
0  
1  
2  
3  
4  
Ciclo completato!
```

Buone pratiche

- Evitare cicli infiniti (es. **while True**) a meno che non siano gestiti con attenzione.
- Usare **for** quando si lavora con sequenze.
- Usare **while** per condizioni dinamiche o input utente.
- Ottimizzare il codice per ridurre il numero di iterazioni inutili.

Collezioni

Python offre diverse strutture dati integrate per gestire collezioni di elementi. Le principali sono **Liste**, **Dizionari**, **Tuple** e **Set**.



1. Liste

Le **liste** sono collezioni ordinate e mutabili. Permettono duplicati.

Creazione

```
lista = [1, 2, 3, 4]
```

Operazioni principali

- **Accesso agli elementi:** `lista[0]` restituisce `1`.
- **Aggiungere elementi:**

```
lista.append(5) # Aggiunge alla fine
lista.insert(1, 10) # Inserisce 10 in posizione 1
```

- **Rimuovere elementi:**

```
lista.remove(3) # Rimuove il primo valore 3 trovato
elemento = lista.pop() # Rimuove e restituisce l'ultimo elemento
```

- **Ordinamento:**

```
lista.sort() # Ordina la lista
lista.reverse() # Ordina in ordine inverso
```

Esempio

```
numeri = [10, 20, 30]
numeri.append(40)
print(numeri) # Output: [10, 20, 30, 40]
```

2. Dizionari

I **dizionari** sono collezioni non ordinate di coppie chiave-valore. Le chiavi sono uniche.

Creazione

```
dizionario = {"nome": "Alice", "età": 25}
```

Operazioni principali

- **Accesso ai valori:**

```
print(dizionario["nome"]) # Output: Alice
```

- **Aggiungere/modificare chiavi:**

```
dizionario["città"] = "Roma" # Aggiunge una nuova coppia chiave-valore
```

- **Rimuovere elementi:**

```
dizionario.pop("età") # Rimuove la chiave "età"
```

- **Iterare sulle chiavi e valori:**

```
for chiave, valore in dizionario.items():  
    print(f"{chiave}: {valore}")
```

Esempio

```
info = {"nome": "Marco", "età": 30}  
info["città"] = "Milano"  
print(info) # Output: {"nome": "Marco", "età": 30, "città": "Milano"}
```

3. Tuple

Le **tuple** sono collezioni ordinate e immutabili. Permettono duplicati.

Creazione

```
tupla = (1, 2, 3, 4)
```

Operazioni principali

- **Accesso agli elementi:**

```
print(tupla[1]) # Output: 2
```

- **Unpacking:**

```
a, b, c, d = tupla
```

- **Conta e indice:**

```
print(tupla.count(2)) # Output: 1  
print(tupla.index(3)) # Output: 2
```

Esempio

```
dati = ("Alice", 25, "Roma")
print(dati[0]) # Output: Alice
```

4. Set

I **set** sono collezioni non ordinate e non permettono duplicati.

Creazione

```
insieme = {1, 2, 3, 4}
```

Operazioni principali

- Aggiungere e rimuovere elementi:

```
insieme.add(5)
insieme.remove(2)
```

- Operazioni insiemistiche:

```
A = {1, 2, 3}
B = {3, 4, 5}
print(A.union(B)) # Output: {1, 2, 3, 4, 5}
print(A.intersection(B)) # Output: {3}
```

Esempio

```
numeri = {1, 2, 3, 3} # I duplicati vengono eliminati
print(numeri) # Output: {1, 2, 3}
```

Confronto tra collezioni

Tipo	Ordinato	Mutabile	Duplicati
Lista	Sì	Sì	Sì
Dizionario	No	Sì (valori)	No (chiavi)
Tupla	Sì	No	Sì
Set	No	Sì	No

3. Python e Intelligenza Artificiale

L'intelligenza artificiale si occupa di creare sistemi in grado di eseguire compiti che normalmente richiedono l'intelligenza umana, come:

- Riconoscimento vocale e visivo.
- Analisi dei dati.
- Previsioni.

Python è ampiamente usato nell'AI grazie alla semplicità del linguaggio e alle potenti librerie disponibili.

Librerie Principali

1. **NumPy**: Per il calcolo scientifico e la manipolazione di array.
 2. **Pandas**: Per l'analisi e la manipolazione dei dati.
 3. **Matplotlib**: Per la visualizzazione dei dati.
 4. **scikit-learn**: Per l'apprendimento automatico.
 5. **TensorFlow** e **PyTorch**: Per reti neurali e deep learning.
 6. **LangChain**: Per costruire applicazioni basate su LLM
-

4. Ambienti di Sviluppo

Un ambiente di sviluppo è fondamentale per scrivere ed eseguire codice. Gli strumenti più utilizzati sono:

- **Jupyter Notebook**: Ideale per il data science e la sperimentazione.
- **PyCharm**: Un IDE completo per lo sviluppo in Python.
- **Visual Studio Code**: Editor di testo leggero con supporto per Python.

in modo particolare utilizzeremo:

- JupyterLab <https://jupyter.org/install>
 - Visual Studio Code <https://code.visualstudio.com/download>
-

5. Esercizi Pratici

Esercizio 1: Creare una lista di numeri e calcolarne la somma

Scrivete un programma che:

1. Crei una lista di numeri.
2. Calcoli e stampi la loro somma.

Esercizio 2: Utilizzare una libreria Python

Installate NumPy e create un array. Stampate la somma di tutti gli elementi.



Esercizio 3: Verifica del numero più grande

Scrivete un programma che prenda due numeri, confronti quale sia maggiore e stampi un messaggio appropriato.

Esercizio 4: Verifica della maggiore età

Scrivete un programma che controlli se una persona è maggiorenne (18 anni o più) e stampi un messaggio appropriato.

Esercizio 5: Parità di un numero

Scrivete un programma che prenda un numero e verifichi se è pari o dispari.

Esercizio 6: Trova il numero più grande tra tre

Scrivete un programma che prenda tre numeri e trovi il più grande tra di loro.